

# Основи API

# API схожий на...



API (інтерфейс прикладного програмування) найкраще розглядати як контракт, який надається одним комп'ютерним програмним забезпеченням іншому.

# Роль API у сучасному програмному забезпеченні

**Інтерфейси прикладного програмування (API)** виконують роль **комунікаційного шару** між різними програмними застосунками, дозволяючи їм **безперешкодно обмінюватися даними та функціональністю**.

Завдяки тому, що API **приховують внутрішню реалізацію** системи, вони забезпечують **сумісність і взаємодію між різнорідними платформами та сервісами**. Це, у свою чергу, **сприяє інноваціям, гнучкості й підвищенню ефективності** у процесі розробки програмного забезпечення.

API допомагають розробникам створювати програми, які приносять користь кінцевому користувачеві



# Типи API

API бувають різних типів, залежно від сфери застосування та рівня доступу:

- **Публічні (Public APIs)** — відкривають сервіси для зовнішніх розробників, дозволяючи інтегрувати функціонал компанії у сторонні застосунки та **розширювати охоплення бренду**.
- **Приватні (Private APIs)** — використовуються **всередині організації** для інтеграції її власних систем і підвищення **внутрішньої ефективності**.
- **Партнерські (Partner APIs)** — надають **контрольований доступ** до певних даних або сервісів між компаніями, що співпрацюють, забезпечуючи **безпечний обмін інформацією**.

Кожен тип API виконує свою роль — від **масштабування бізнесу** до **оптимізації внутрішніх процесів**.



# Публічний API

**Публічні API** (також відомі як **зовнішні** або **відкриті API**) — це інтерфейси, які надають **доступ до сервісів або даних провайдера** стороннім розробникам та зовнішнім користувачам.

Їхня мета — **розширити функціональність зовнішніх застосунків, стимулювати інновації та створювати нові бізнес-можливості.**

Такі API зазвичай:

- мають **ретельно підготовлену документацію**;
- супроводжуються **підтримкою для розробників**;
- розробляються з акцентом на **зручність і простоту інтеграції**.

Втім, керування публічним API вимагає суворих заходів безпеки, таких як:

- **контроль доступу та автентифікація**,
- **лімітування (throttling) запитів**,
- **контроль версій**.

Це необхідно для забезпечення **стабільності та безпеки** роботи в умовах великої кількості користувачів і запитів.

# Приватний API

**Приватні API** (або **внутрішні API**) створюються для **використання всередині організації**. Вони забезпечують **безпечний обмін даними** між різними командами, підрозділами чи системами компанії.

На відміну від публічних, приватні API **не відкриваються зовнішнім користувачам**. Їх основна мета — **підвищення продуктивності, ефективності та спрощення інтеграції** внутрішніх програмних рішень.

Використання приватних API дозволяє:

- **оптимізувати бізнес-процеси,**
- **автоматизувати рутинні операції,**
- **забезпечити збереження конфіденційних даних у межах безпечної IT-інфраструктури організації.**

Таким чином, приватні API сприяють **гнучкості та узгодженій роботі** всіх внутрішніх систем підприємства.

# Партнерський API

Партнерські API спеціально розроблені для стратегічних бізнес-партнерів і забезпечують більш контрольований доступ порівняно з публічними API. Вони дозволяють безпосередню інтеграцію між організацією та обраними зовнішніми сторонами, сприяють тіснішій співпраці та надають взаємні переваги, такі як спільні сервіси, обмін даними та створення додаткової цінності. Доступ до партнерських API зазвичай регулюється суворими контрактними угодами, що забезпечує безпеку та відповідність стандартам, одночасно підтримуючи довірчу партнерську екосистему.



# Композитний API

Композитні API об'єднують кілька викликів API в один, спрощуючи взаємодію між клієнтом і сервером, особливо під час виконання складних операцій. Такий підхід зменшує навантаження на мережу, підвищує швидкість отримання даних і покращує користувацький досвід, мінімізуючи кількість запитів і відповідей між клієнтом і сервером. Композитні API особливо корисні в архітектурах мікросервісів та веб-додатках, де потрібно координувати роботу кількох сервісів, забезпечуючи більш ефективний та узгоджений метод обробки даних.

# RESTful API vs. SOAP API

**RESTful API та SOAP (Simple Object Access Protocol) API представляють два основні підходи до веб-сервісів. RESTful API, орієнтовані на веб-ресурси та безстанову (stateless) взаємодію, пропонують простоту та гнучкість. Натомість SOAP API, визначені суворими стандартами та повідомленнями на основі XML, забезпечують високий рівень безпеки та надійності транзакцій.**



# GraphQL: Альтернатива RESTful

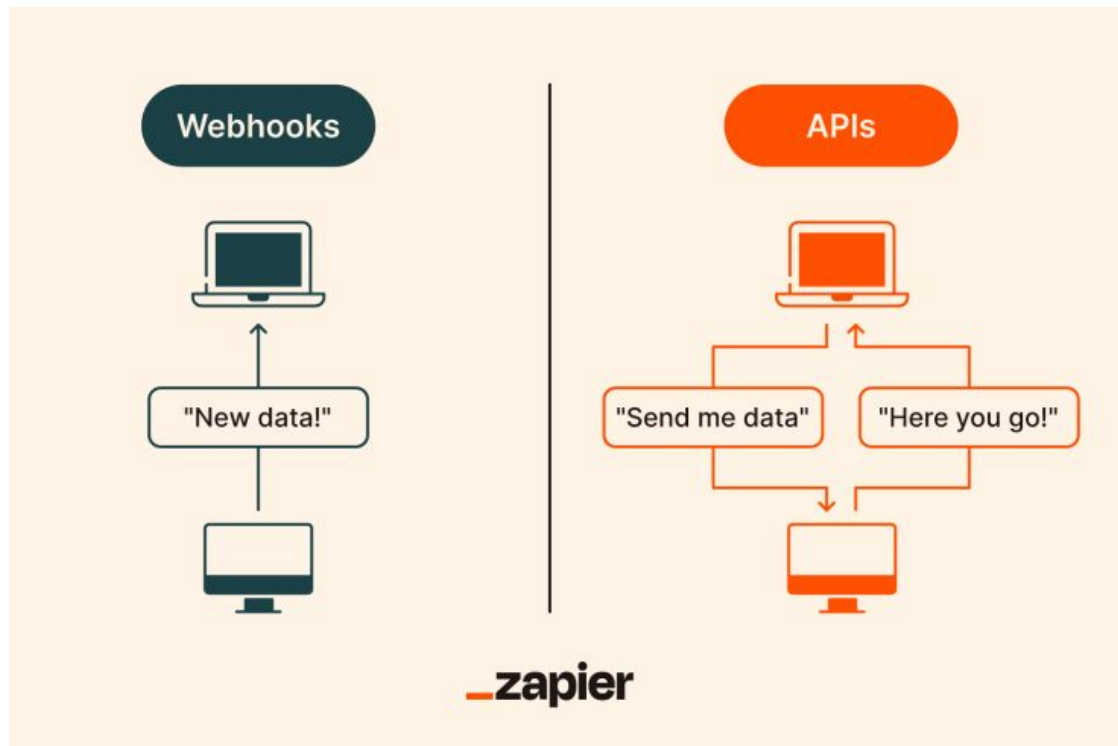
GraphQL пропонує альтернативу RESTful API, дозволяючи клієнтам запитувати саме ті дані, які їм потрібні, зменшуючи надмірне або недостатнє отримання інформації. Ця мова запитів для API забезпечує більш ефективний, потужний і гнучкий підхід до отримання даних, задовольняючи динамічні потреби сучасних додатків.



# Розуміння Webhooks

**Webhooks — це визначені користувачем HTTP-зворотні виклики, які надають механізм для отримання додатками повідомлень у реальному часі про події в інших сервісах. На відміну від API, які потребують опитування для отримання даних, webhooks відправляють дані на вказану URL-адресу у відповідь на певні тригери, підвищуючи ефективність і оперативність інтеграції додатків.**

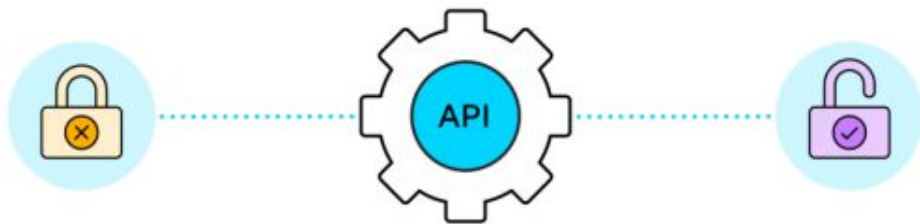
# Webhooks vs APIs



# Основи автентифікації API

Автентифікація API перевіряє особу виклику, забезпечуючи, що запит робиться від відомого користувача чи додатка. До поширених методів належать:

- **Basic Authentication** — використання імені користувача та пароля;
- **API-ключі** — унікальні ідентифікатори, надані додаткам;
- більш складні протоколи, такі як **OAuth**, для делегованої авторизації.



# OAuth детально

OAuth — це широко вживаний протокол, який дозволяє стороннім сервісам обмінюватися веб-ресурсами від імені користувача. Він забезпечує безпечний та ефективний спосіб надання додаткам доступу до інформації користувача без розкриття пароля, використовуючи токени для представлення наданої авторизації.

# Впровадження JWT для безпечного доступу до API

JSON Web Tokens (JWT) пропонують компактний, безпечний для URL спосіб представлення тверджень (claims), які передаються між двома сторонами. У сфері безпеки API JWT використовуються для кодування інформації про користувача, яку сервер перевіряє для авторизації доступу, що робить їх особливо корисними для **single sign-on (SSO)** та аутентифікації на основі токенів.



# API-ключі: використання та безпека

API-ключі слугують простими ідентифікаторами для контролю доступу і зазвичай використовуються у публічних API для ідентифікації додатка, який здійснює виклик. Хоча їх легко впровадити, їх слід захищати від розкриття, щоб запобігти несанкціонованому використанню, зазвичай шляхом зберігання поза кодом та використання безпечних рішень для зберігання.

# Розуміння Access Tokens та Refresh Tokens

**Access tokens** — це короткострокові токени, які використовуються для виконання аутентифікованих запитів до API.

**Refresh tokens** — токени з тривалішим терміном дії, які використовуються для отримання нових access tokens.

Цей механізм зменшує ризик крадіжки токенів та забезпечує користувачам можливість підтримувати сесії без повторного входу в систему.

# Захист API за допомогою SSL/TLS

Шифрування трафіку між клієнтом і сервером за допомогою SSL/TLS є базовим заходом безпеки для API. Воно забезпечує захист обміну даними від прослуховування та підробки, що є обов'язковим при передачі конфіденційної інформації, такої як токени аутентифікації, через інтернет.



# Найкращі практики безпеки API

Окрім автентифікації та авторизації, захист API потребує комплексного підходу, що включає:

- перевірку та очищення вхідних даних;
- правильну обробку помилок, щоб уникнути витоку інформації;
- регулярний аудит та тестування API на вразливості, щоб підтримувати заходи безпеки актуальними та ефективними.

# Обмеження та регулювання швидкості запитів до API

**Rate limiting** та **throttling** є важливими для контролю навантаження на API та запобігання його зловживанню. Контролюючи кількість запитів, які користувач може зробити протягом певного часу, ці практики допомагають підтримувати надійність і продуктивність API, забезпечуючи справедливе використання та доступність для всіх споживачів.

# Обмеження та регулювання швидкості запитів до API

- API-запит складається з кількох ключових компонентів:
  - **URL-endpoint** — представляє ресурс;
  - **HTTP-method** — вказує дію (наприклад, GET, POST, PUT, DELETE);
  - **Заголовки (headers)** — містять метадані;
  - **Тіло запиту (request body)** — містить дані, що надсилаються, для запитів POST і PUT.

Розуміння цих елементів є критично важливим для ефективної взаємодії з API.



The diagram illustrates the components of an API request. On the left, four labels are listed: **Endpoint**, **HTTP Method**, **HTTP Headers**, and **Body**. Arrows point from each label to its corresponding value in a code block on the right. The **Endpoint** is `https://apiurl.com/review/new`, the **HTTP Method** is `POST`, the **HTTP Headers** include `content-type: application/json`, `accept: application/json`, and `authorization: Basic abase64string`, and the **Body** is a JSON object: `{ "review" : { "title" : "Great article!", "description" : "So easy to follow.", "rating" : 5 } }`.

```
Endpoint — https://apiurl.com/review/new
HTTP Method — POST
HTTP Headers — content-type: application/json
               accept: application/json
               authorization: Basic abase64string
Body — {
        "review" : {
          "title" : "Great article!",
          "description" : "So easy to follow.",
          "rating" : 5
        }
      }
```

# Інтерпретація відповідей API

Відповіді API містять **статус-коди**, **заголовки (headers)** та **вміст тіла відповіді (body)**, надаючи зворотний зв'язок про результат запиту.

- **Статус-коди**, такі як 200 OK або 404 Not Found, вказують на успіх або невдачу запиту.
- **Тіло відповіді**, зазвичай у форматі JSON або XML, містить запитані дані або деталі помилки, яка

```
# Request
curl -H "Authorization: Bearer <personal_access_token>" "https://app.asana.com/api/1.0/tasks/1224?opt_fields=name,notes"

# Response
HTTP/1.1 200
{
  "data": {
    "name": "Make a list",
    "notes": "Check it twice!",
    "id": 1224
  }
}
```

# Важливість документації API

Повна документація API є критично важливою для залучення розробників та ефективного використання API. Вона повинна містити деталі доступних **ендпоінтів**, формати **запитів/відповідей** та вимоги до **автентифікації**, а також надавати приклади з реального використання. Хороша документація служить як **посібником**, так і **довідником** для розробників, полегшуючи процес інтеграції.



# Стратегії версіонування API

Версіонування API є важливим для управління змінами та забезпечення зворотної сумісності. Стратегії, такі як **версіонування через URI-путь, параметри рядка запиту або кастомні заголовки запиту**, дозволяють розробникам впроваджувати нові функції чи зміни без порушення існуючих інтеграцій, підтримуючи контракт з користувачами API.

# Майбутнє API

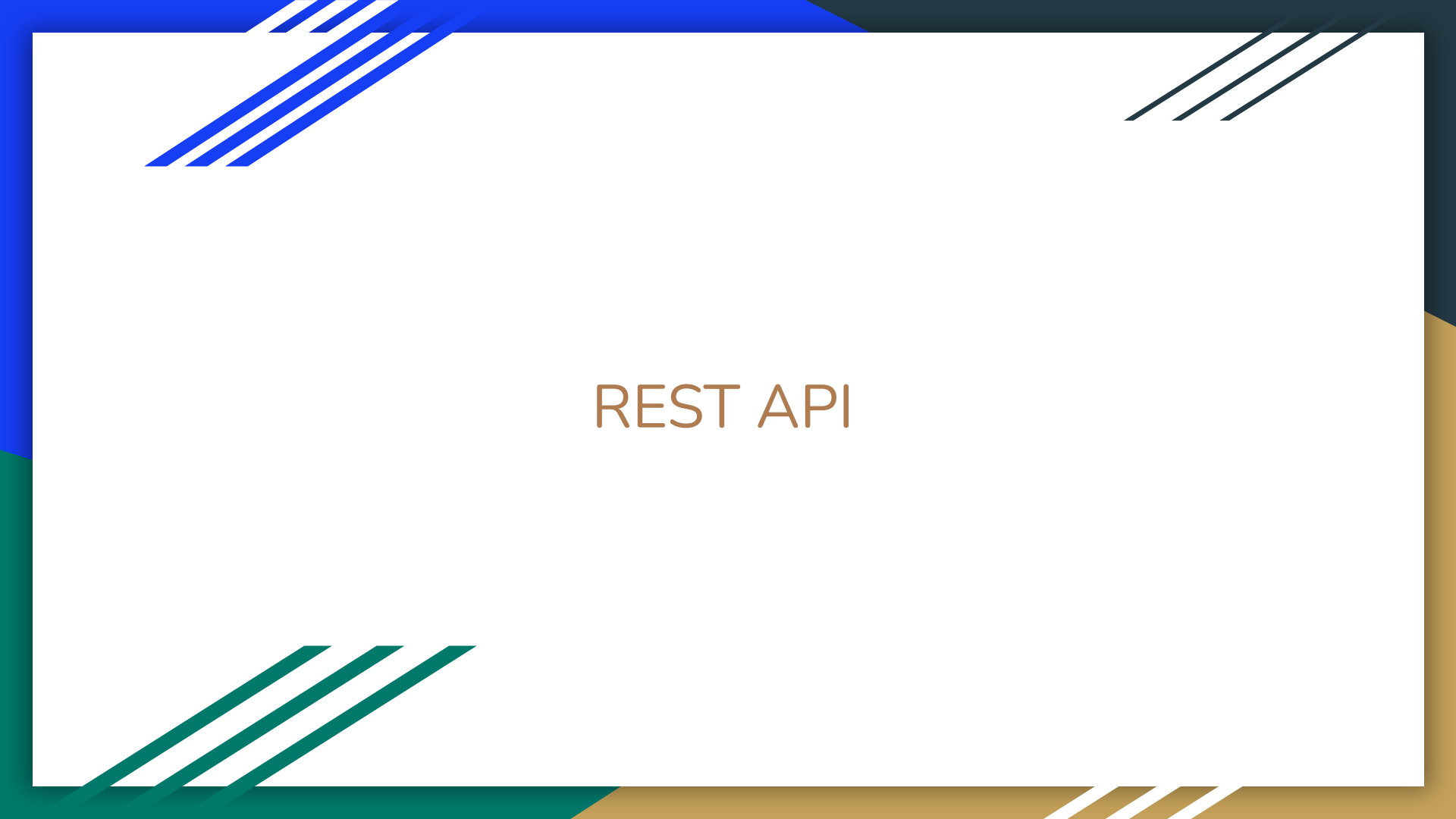
Еволюція API продовжує формувати цифровий ландшафт, з тенденціями до більш **децентралізованих підходів**, таких як мікросервіси, та впровадження **штучного інтелекту та машинного навчання** для розширення функціональності API. Майбутнє API полягає в їх здатності адаптуватися, пропонуючи більш **персоналізовані, інтелектуальні та автономні можливості**, щоб задовольнити зростаючі потреби технологічного розвитку.

# Тестування API та найкращі практики

Ефективне тестування API забезпечує їхню роботу відповідно до очікувань, охоплюючи **функціональність, надійність, продуктивність та безпеку**. Найкращі практики включають:

- **детальне планування тестів,**
- **автоматизацію,**
- **управління середовищем,**
- **безперервний моніторинг.**

Забезпечення якості на всьому життєвому циклі API є необхідним для надання **надійних та стійких сервісів**.



# REST API

# Вступ до RESTful API

RESTful API є основою веб-комунікацій, дозволяючи різним програмним системам обмінюватися даними через Інтернет у зручний та безшовний спосіб. Дотримуючись принципів REST, ці API підтримують створення веб-сервісів, які є **масштабованими, надійними та простими у використанні**, що робить їх невід'ємною частиною сучасної веб-розробки та інтеграції додатків.

# REST API

REST API визначає набір функцій, за допомогою яких розробники можуть виконувати запити та отримувати відповіді через протокол HTTP, наприклад GET і POST.

REST найкраще пояснюється як спосіб спілкування з іншими машинами через сервер за допомогою протоколу HTTP за допомогою *дієслів (дії)* та *іменників (елементи)*

# Розуміння REST

Representational State Transfer (REST) — це архітектурний стиль, розроблений для розподілених систем, особливо для вебу. Він підкреслює **простоту, масштабованість та продуктивність**. RESTful-системи працюють через HTTP-запити для доступу та використання даних. Принципи REST, включаючи **безстановість (statelessness), уніфікований інтерфейс, кешованість та багаторівневу систему**, керують розробкою веб-сервісів, які легко підтримувати та масштабувати.

# REST API vs. RESTful API: розуміння різниці

Терміни **REST API** та **RESTful API** часто використовуються як синоніми в індустрії, проте вони підкреслюють різні аспекти одного й того ж поняття. **REST** (Representational State Transfer) — це архітектурний стиль, визначений шістьма керівними обмеженнями для створення веб-сервісів. **REST API** — це будь-який API, який дотримується цих архітектурних обмежень REST.

Термін **RESTful** більше є кваліфікатором, що вказує на те, що API дійсно втілює та дотримується принципів REST. По суті, всі RESTful API є REST API, але не всі REST API повністю дотримуються принципів REST, щоб їх можна було вважати RESTful.

Це розмежування підкреслює важливість дотримання принципів REST — включаючи **безстановість (statelessness)**, **архітектуру клієнт-сервер**, **кешованість**, **багаторівневу систему**, **код на вимогу (опційно)** та **уніфікований інтерфейс** — при розробці справді RESTful сервісів.



# Ключові характеристики RESTful API

RESTful API визначаються такими характеристиками, як **безстановість (statelessness)**, коли кожен запит від клієнта до сервера має містити всю інформацію, необхідну для розуміння та виконання запиту.

Вони також дотримуються **уніфікованого інтерфейсу**, що забезпечує стандартизований спосіб взаємодії між клієнтом і сервером.

Така уніфікованість дозволяє **роз'єднувати архітектуру**, що підвищує масштабованість і продуктивність веб-сервісів.

# CRUD-операції у RESTful API

**CRUD-операції** є основою взаємодії з RESTful API, відображаючи базові дії, які виконують бази даних над даними: **Create (створити)**, **Read (читати)**, **Update (оновити)** та **Delete (видалити)**.

У контексті RESTful API ці операції безпосередньо відповідають **HTTP-методам**, забезпечуючи стандартизований спосіб обробки ресурсів:

- **Create** використовує метод **POST** для створення нових ресурсів.
- **Read** отримує дані за допомогою методу **GET**, роблячи запити **ідемпотентними та безпечними**, тобто кілька запитів мають той самий ефект, що й один.
- **Update** змінює існуючі ресурси і може здійснюватися через методи **PUT** або **PATCH**, де **PUT** замінює весь ресурс, а **PATCH** застосовує часткові оновлення.
- **Delete** видаляє ресурси за допомогою методу **DELETE**.

Таке відповідність забезпечує **зрозумілий та інтуїтивний спосіб управління веб-ресурсами**, дозволяючи розробникам виконувати операції, подібні до транзакцій баз даних, через Інтернет з легкістю та точністю.



# Ідентифікація ресурсів у REST

- У RESTful API ресурси ідентифікуються за допомогою **уніфікованих ідентифікаторів ресурсів (URI)**, зазвичай у вигляді URL.
- Добре спроектований REST API забезпечує **інтуїтивний та послідовний доступ до ресурсів**, використовуючи URI для представлення окремих сутностей або колекцій сутностей у **ієрархічній структурі**.
- Такий підхід гарантує, що взаємодія з API є **прямолінійною та орієнтованою на ресурси**.

# Безстановість у RESTful архітектурі

- **Безстановість** RESTful API означає, що кожен запит від клієнта до сервера має містити всю інформацію, необхідну серверу для виконання запиту.
- Сервер **не зберігає стан сесії клієнта** на своїй стороні.
- Цей принцип проектування спрощує архітектуру сервера, підвищує масштабованість та покращує **надійність і продуктивність**.

# Формати відповідей RESTful API

RESTful API зазвичай використовують **JSON (JavaScript Object Notation)** або **XML (Extensible Markup Language)** як мову обміну даними, що забезпечує ефективний та незалежний від платформи обмін інформацією.

**JSON**, зокрема, здобув популярність завдяки своїй **легкості та простоті використання з JavaScript**, що робить його переважним вибором для веб-додатків.

# Міркування щодо безпеки RESTful API

Захист RESTful API є критично важливим для **захисту конфіденційних даних** та забезпечення **безпечної взаємодії між клієнтом і сервером**.

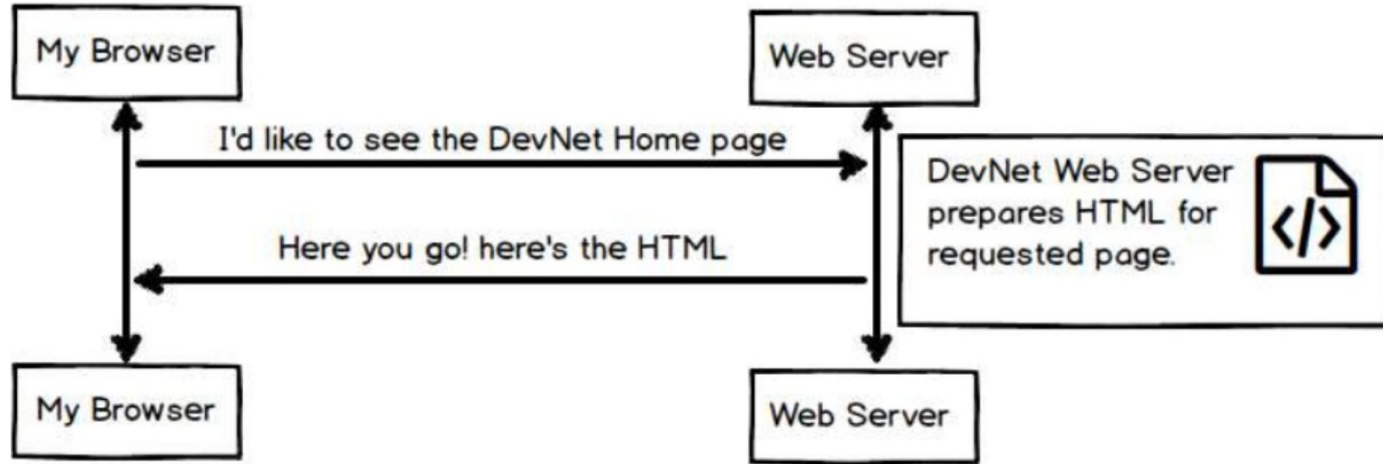
Поширені практики безпеки включають:

- використання **HTTPS** для шифрованого обміну даними;
- впровадження **токенів автентифікації**;
- застосування механізмів **контролю доступу** для обмеження доступу до ресурсів.

Крім того, **перевірка вхідних даних** та **регулярне тестування безпеки** є необхідними для зменшення вразливостей.

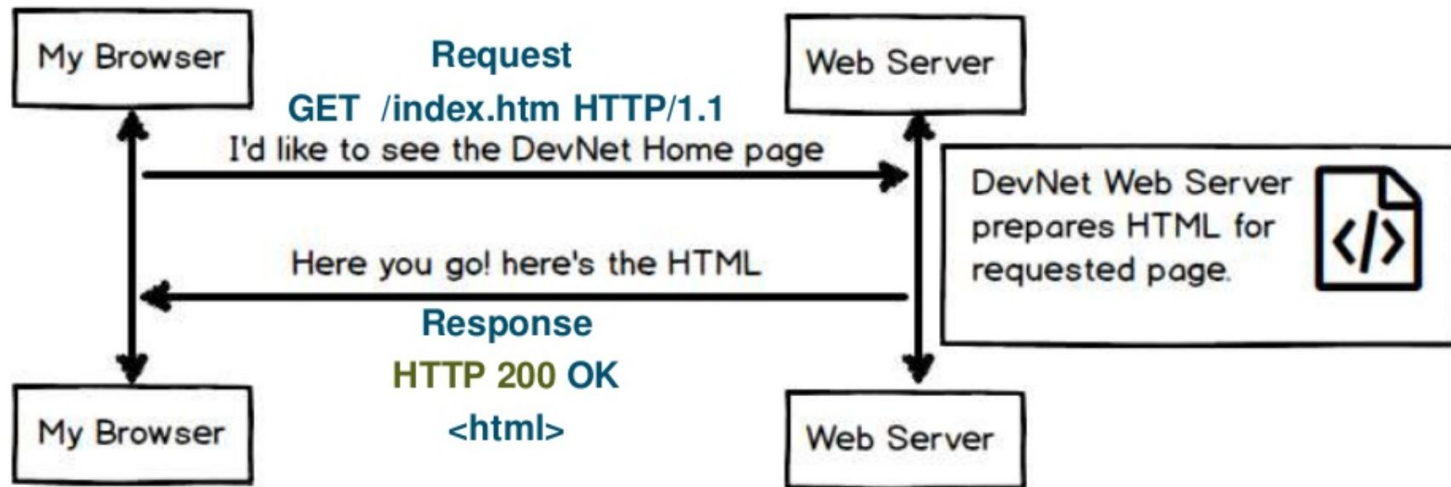
# Знайте, що таке запити та відповіді

## View a Web Page



# Знайте, що таке запити та відповіді

View a Web Page

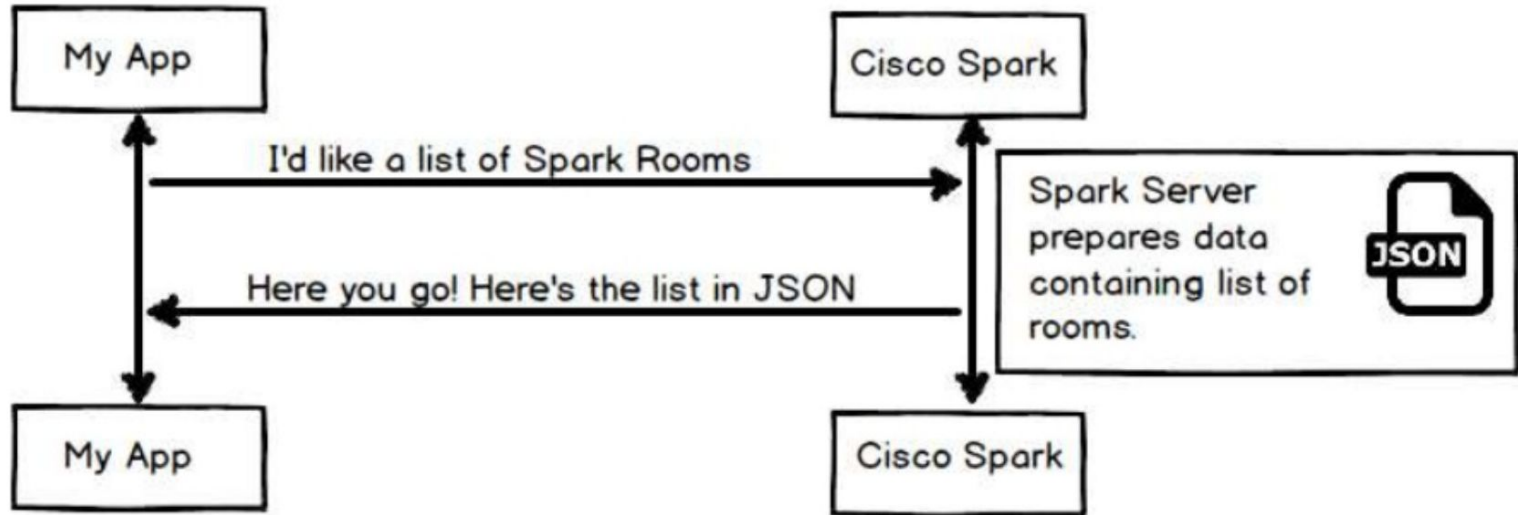




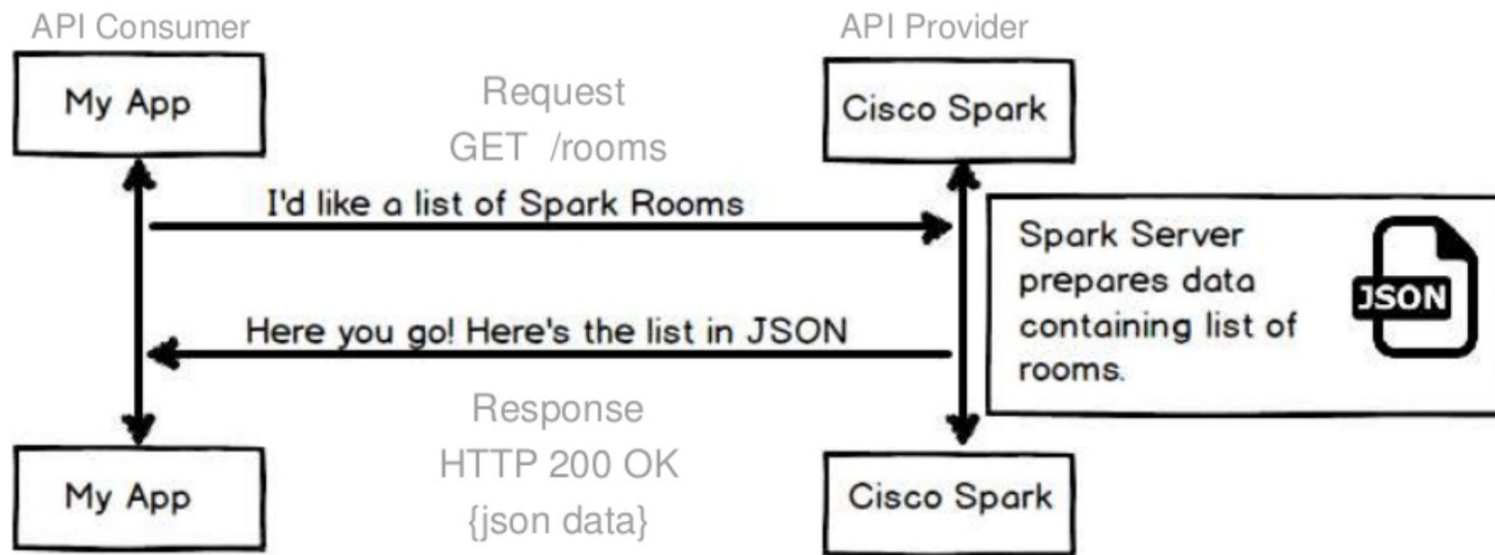
REST API також використовують Request і Response



## Отримання даних за допомогою API (Spark)



# Отримайте дані за допомогою API – відповідь!



# Список дієслів!

## List items

GET /items

## Create an item

POST /items

## Get an item

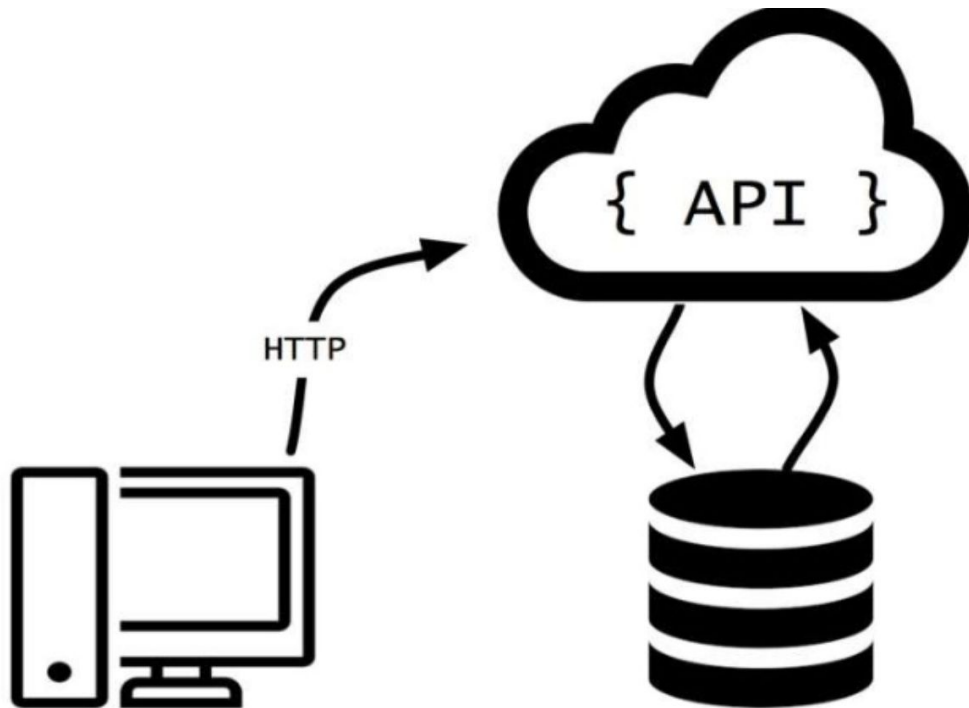
GET /items/{id}

## Update an item

PUT /items/{id}

## Delete an item

DELETE /items/{id}



# Документація – це ваш словник

Довідник API для кімнат (дієслова та іменники)



Spark for Developers

Documentation

Blog

Support

Haus

Fun

## API REFERENCE

People

Rooms

Memberships

Messages

Teams

Team Memberships

Webhooks

Organizations

Licenses

POST

<https://api.ciscospark.com/v1/people>

Create a Person

GET

<https://api.ciscospark.com/v1/people/{personId}>

Get Person Details

PUT

<https://api.ciscospark.com/v1/people/{personId}>

Update a Person

DELETE

<https://api.ciscospark.com/v1/people/{personId}>

Delete a Person

GET

<https://api.ciscospark.com/v1/people/me>

Get My Own Details

## Rooms

Rooms are virtual meeting places where people post messages and collaborate to get work done. This API is used to manage rooms themselves. Rooms are created and deleted with this API. You can also update a room to change its title, for example.

## Параметри запиту для “MESSAGES”

### Request Parameters

| Name          | Type   | Description                                                                                                                                                                      |
|---------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| roomId        | string | The room ID.                                                                                                                                                                     |
| toPersonId    | string | The ID of the recipient when sending a private1:1 message.                                                                                                                       |
| toPersonEmail | string | The email address of the recipient when sending a private 1:1 message.                                                                                                           |
| text          | string | The message, in plain text. If <b>markdown</b> is specified this parameter may be <i>optionally</i> used to provide alternate text for UI clients that do not support rich text. |
| markdown      | string | The message, in markdown format.                                                                                                                                                 |

# Проектування RESTful API

Проектування RESTful API вимагає **ретельного планування** та дотримання принципів REST.

Цей процес включає:

- визначення **назв ресурсів**;
- визначення **відносин між ресурсами**;
- вибір **відповідних HTTP-методів** для дій.

Ефективне проектування API сприяє **зручності використання, підтримованості та масштабованості**, забезпечуючи відповідність API потребам як розробників, так і кінцевих користувачів.

# Інструменти



# Дізнайтеся, як використовувати інструменти налагодження/тестування для API

## Інструменти для налагодження

- Налаштування Webhook (раніше RequestBin – [requestbin.org](https://requestbin.org)...)
- Утиліти Webhook (Torpio...)
- Місцеве тунелювання (ngrok...)
- Моніторинг API (Runscope...)
- Насмішка відповіді (mocky.io...)
- Утиліти JSON (JSONFormat...)
- Утиліти OAUTH (oauth.io...)
- Каталоги API (APIS.io, ProgrammableWeb...)
- Тестування API (Runscope Radar...)
- Тестування навантаження (loader.io...)
- Клієнти GUI HTTP (POSTMAN...)

# POSTMAN



Spark stsftartz

List DevNet repos (JSON)

method → GET

url → https://api.github.com/users/CiscoDevNet/repos?page=1&per\_page=2

query parameters → Params

request headers → Headers (1)

response body → Body

response headers → Headers (23)

status code → Status: 200 OK

content-type → JSON

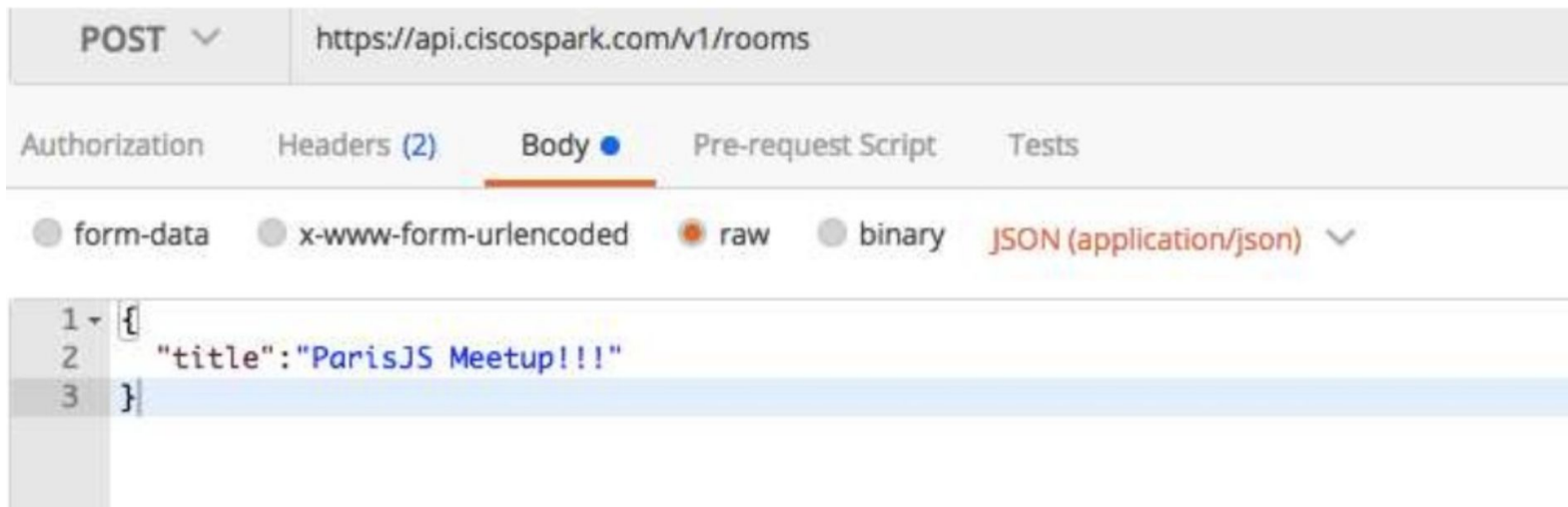
request headers table:

| key    | value            |
|--------|------------------|
| Accept | application/json |

response body table:

|   |   |
|---|---|
| 1 | { |
| 2 |   |

# Додайте JSON до тіла в POSTMAN



# Питання та відповіді